INDIAN INSTITUTE OF TECHNOLOGY BOMBAY

WINTER IN DATA SCIENCE 5.0

Organised by

Analytics Club, IITB

ENDTERM REPORT

BLOCKCHAIN INTEGRATED MACHINE LEARNING SYSTEM

FOR CYPTOCURRENCY FRAUD DETECTION

Authored by:

DEV KUMAR

23B2529

INDEX

Data Analysis

A complete data analysis pipeline was carried out to transform the raw dataset into a clean, structured, and information-rich form suitable for further modelling. The objective was not only to preprocess the data mechanically, but also to understand its statistical behaviour and ensure that each transformation added value rather than distortion. The pipeline consisted of data cleaning, missing value treatment, outlier handling, encoding, feature scaling, and dimensionality reduction using Principal Component Analysis (PCA).

Data Cleaning

Raw datasets often contain inconsistencies such as duplicate entries, incorrect data types, and noisy column formats. If left unaddressed, these issues can lead to biased learning and misleading statistical inferences.

Methodology

- Duplicate records were identified and removed.
- Column names were standardized for consistency.
- Data types were verified and corrected to ensure numerical and categorical features were properly represented.

Handling Missing Values

Missing values can arise due to data collection errors, sensor failures, or incomplete user inputs. Blindly removing missing values can lead to significant information loss, while poor imputation can distort distributions.

Median is preferred when the data is skewed or contains outliers.

Methodology

- Numerical features: median imputation
- Categorical features: mode imputation

Outlier Detection and Treatment

Outliers are extreme observations that deviate significantly from the rest of the data. The **Interquartile Range (IQR)** method defines outliers as:

$$LowerBound = Q_1 - 1.5 \times IQR$$

$$UpperBound = Q_3 + 1.5 \times IQR$$

where:

$$IQR = Q_3 - Q1$$

Methodology

- Outliers were detected using the IQR rule.
- Data points outside acceptable bounds were removed after verification.

Encoding of Categorical Variables

Machine learning algorithms operate on numerical data. Categorical variables must be transformed without introducing false relationships.

- **Ordinal features** → Label Encoding
- **Nominal features** → One-Hot Encoding

Feature Scaling

Feature scaling ensures that all features contribute equally. **Standardization** transforms data as:

$$z = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation.

Principal Component Analysis (PCA)

PCA reduces dimensionality by projecting data onto orthogonal directions of maximum variance. Each principal component is an eigenvector of the covariance matrix.

$$\sum = \frac{1}{n} X^T X$$

The eigenvalues indicate the variance captured by each component.

Methodology

- PCA was applied after encoding and scaling.
- Number of components was chosen using cumulative explained variance.

Bayesian Classification

Bayesian classification is a probabilistic approach to classification that relies on Bayes' theorem to estimate the likelihood of a class given observed feature. Instead of directly learning decision boundaries, Bayesian methods model the underlying data distribution.

Bayes' Theorem and Classification Rule

Bayes' theorem allows us to compute the posterior probability of a class C_x given a feature vector x :

$$P(C_k \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid C_k)P(C_k)}{P(\mathbf{x})}$$

For classification, the denominator P(x) is constant across classes and can be ignored. The decision rule becomes:

$$\hat{C} = \arg\max_k P(\mathbf{x} \mid C_k)P(C_k)$$

Intuition

- $P(C_k)$: Prior belief about the class
- $P(x \nmid C_k)$: Likelihood of observing the data given the class
- Choose the class that best explains the observed data

Naive Bayes Classifier

Naive Bayes assumes **conditional independence** between features:

$$P(\mathbf{x} \mid C_k) = \prod_{i=1}^{d} P(x_i \mid C_k)$$

While this assumption is often violated in practice, the classifier performs surprisingly well for many real-world problems.

Why It Works Well in ML

- Extremely fast to train
- Works well with high-dimensional data
- Robust to irrelevant features

Support Vector Machines (SVM)

Support Vector Machines are margin-based classifiers that aim to find the **optimal separating hyperplane** between classes.

Given labelled training data, SVM finds the decision boundary that maximizes the margin, i.e., the distance between the closest data points (support vectors) and the boundary.

Linear SVM Decision Function

$$f(x) = w^{\mathrm{T}}x + b$$

Classification is based on the sign of $f(x)$.

Soft Margin and Regularization

To handle non-separable data, SVM introduces slack variables and a regularization parameter C , controlling the trade-off between margin maximization and misclassification.

Why SVM is Powerful

- Effective in high-dimensional spaces
- Robust to overfitting with proper regularization
- Works well with small and medium datasets

Decision Trees

Decision Trees classify data by recursively splitting it based on feature values to maximize class purity.

Entropy and Information Gain

Entropy

$$H(Y) = -\sum_{k} p_k \log_2 p_k$$

Entropy measures uncertainty in class labels.

Information Gain

$$IG(Y, X) = H(Y) - H(Y \mid X)$$

The feature with the highest information gain is chosen for splitting.

Tree Construction

- Start with the full dataset at the root
- Select the best feature based on information gain
- Split data and repeat recursively
- Stop when purity is high or constraints are met

Advantages

- Easy to interpret
- Handles non-linear relationships
- Requires minimal preprocessing

Limitations

- Prone to overfitting
- Sensitive to small data changes

Univariate statistics

Univariate statistics describe the behaviour of **individual features** in a dataset. Understanding feature-level distributions helps in detecting skewness, outliers, and scaling requirements before model training.

Key Statistical Measures

Let $X = \{x_1, x_2, \dots, x_n\}$ be a feature.

- **Mean**

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

- **Variance**

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n} (x_i - \mu)^2$$

- **Standard Deviation**

$$\sigma = \sqrt{\sigma^2}$$

- **Median**: Robust measure of central tendency
- **Skewness**: Indicates asymmetry in distribution

Density Estimation

Density estimation is the process of estimating the **probability distribution** that generated the observed data.

In ML, density estimation is useful for:

- Probabilistic classification (Naive Bayes)
- Anomaly detection
- Understanding data structure

Parametric Density Estimation

Parametric methods assume that data follows a known distribution with a fixed number of parameters.

For example, a **Gaussian distribution** is defined by mean and variance:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Non-Parametric Density Estimation

Non-parametric methods do not assume a predefined distribution. KDE estimates density by placing kernels (e.g., Gaussians) over data points.

$$\hat{p}(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)$$

where:

- h is the bandwidth
- $K(.)$ is the kernel function

Bias–Variance Perspective

Although not explicitly model-specific, understanding bias and variance explains why different models behave differently.

- **High Bias**: Model too simple (e.g., Naive Bayes)
- **High Variance**: Model too complex (e.g., deep Decision Trees)

Machine Learning Models

Logistic Regression

Logistic Regression is a linear probabilistic classifier that models the conditional probability of a binary outcome given input feature. Despite its simplicity, it serves as a strong baseline for many classification tasks.

The model computes a linear combination of features and passes it through the sigmoid function:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

Where:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Model parameters are learned by minimizing the **log loss** using gradient-based optimization.

Practical Behavior

- Produces smooth decision boundaries
- Outputs well-calibrated probabilities
- Sensitive to feature scaling
- Performs well when classes are linearly separable

Random Forest Classifier

Random Forest is an ensemble of decision trees built using bootstrapped datasets and random feature selection.

Each tree learns independently, and final predictions are obtained via majority voting.

Practical Behaviour

- Reduces overfitting compared to single trees
- Robust to noise
- Handles mixed feature types well

XGBoost Classifier

XGBoost is a gradient-boosted tree ensemble that builds trees sequentially to correct previous errors.

Each tree minimizes a regularized loss function:

$$\mathcal{L} = \sum \ell(y_i, \hat{y}_i) + \Omega(f)$$

Practical Behaviour

- High accuracy on tabular data
- Strong regularization
- Efficient handling of missing values

Blockchain

Introduction

A blockchain is a **distributed, append-only digital ledger** that records transactions across a network of computers in a secure, transparent, and tamper-resistant manner. Unlike traditional centralized databases, a blockchain does not rely on a single trusted authority. Instead, trust is established through cryptography, consensus mechanisms, and decentralization.

Each record in a blockchain is stored in a **block**, and blocks are linked together using cryptographic hashes, forming a chronological chain. Once data is added to the blockchain, it becomes extremely difficult to alter without detection.

Key characteristics of blockchain systems include:

- **Decentralization**: No single entity controls the ledger
- **Immutability**: Past records cannot be changed
- **Transparency**: Transactions are publicly verifiable
- **Security**: Cryptographic techniques protect data integrity

Blockchain Technology

At a high level, a blockchain system consists of the following components:

Blocks

Each block typically contains:

- A list of transactions
- A timestamp
- A cryptographic hash of the previous block
- A nonce (used in mining, where applicable)

Chain Structure

Blocks are linked together by storing the **hash of the previous block** inside the current block. This creates a dependency such that altering one block invalidates all subsequent blocks.

Distributed Network

The blockchain ledger is replicated across multiple nodes in a peer-to-peer network. Each node independently verifies transactions and maintains a copy of the ledger.

Consensus Mechanism

Consensus protocols (e.g., Proof of Work) ensure that all nodes agree on the state of the blockchain without central coordination.

This architecture ensures fault tolerance, transparency, and resistance to single-point failures.

Blockchain and Cryptography

A hash function maps input data of arbitrary size to a fixed-size output. Blockchain systems rely heavily on **SHA-256**, a secure hash algorithm.

Properties of SHA-256:

- **Deterministic**: Same input always produces the same output
- **Collision resistant**: Extremely unlikely for two inputs to have the same hash
- **Preimage resistant**: Impossible to recover input from hash
- **Avalanche effect**: Small input changes cause large output changes

These properties allow hashes to act as **digital fingerprints** for data.

Role of SHA-256 in Blockchain

SHA-256 is used to:

- Link blocks together securely

- Detect any tampering with past data
- Secure transactions and block headers
- Enable mining through computational puzzles

If any transaction in a block changes, its hash changes, breaking the chain and alerting the network.

Public Key Cryptography

Blockchain systems use asymmetric cryptography:

- **Private keys**: Used to sign transactions
- **Public keys**: Used to verify signatures

This ensures that only the owner of a private key can authorize transactions from a given address.

Smart Contracts

Smart contracts are **self-executing programs** stored on the blockchain that automatically enforce rules and conditions.

Key Characteristics:

- Deterministic execution
- Immutable after deployment
- Transparent and verifiable
- No intermediaries required

Smart contracts enable decentralized applications (DApps) by embedding logic directly into the blockchain. Examples include escrow services, voting systems, and digital asset management.

Ethereum

Ethereum extends blockchain functionality beyond simple transactions by introducing a **Turing-complete virtual machine**, known as the Ethereum Virtual Machine (EVM).

Key features of Ethereum:

- Supports smart contracts
- Uses Solidity as its primary programming language
- Allows decentralized application development
- Maintains state across contract executions

Ethereum transforms the blockchain from a ledger into a **global decentralized computing platform**.

Bitcoin Architecture

Bitcoin is the first successful implementation of blockchain technology and serves as the foundational model for decentralized systems.

Transaction Model

Bitcoin uses a **UTXO (Unspent Transaction Output)** model, where transactions consume previous outputs and create new ones.

Block Structure

Each Bitcoin block contains:

- Block header (previous hash, nonce, timestamp)
- Merkle root summarizing all transactions
- List of transactions

Mining and Proof of Work

Miners compete to solve a computational puzzle by finding a nonce such that the block hash meets a difficulty target.

This process:

- Secures the network
- Prevents double spending
- Enables decentralized consensus

Importance of SHA-256 in Bitcoin

SHA-256 is used extensively in Bitcoin:

- Hashing block headers
- Creating Merkle trees
- Mining via Proof of Work
- Securing transaction history

The computational cost of reversing or altering SHA-256 hashes makes Bitcoin's blockchain practically immutable.

FINAL PROJECT

Fraud Detection Problem Definition

The problem was formulated as a **binary classification task**, where Ethereum transactions are classified as:

- **Legitimate (0)**
- **Fraudulent / Erroneous (1)**

The dataset provided an isError field, which was treated as the **ground truth fraud label**. Since raw blockchain data is not directly suitable for ML, the focus was shifted from transaction identifiers to **behavioural transaction patterns**.

Machine Learning Approach

Instead of using raw fields such as wallet addresses or transaction hashes (which are non-numeric and high cardinality), I engineered **aggregated behavioural features** that better represent fraud patterns:

Feature	Rationale
`tx_count`	Fraudulent wallets often show abnormal transaction frequency
`avg_value`	Scam transactions tend to have atypical average values
`total_value`	Sudden high-value transfers indicate possible fraud
`unique_receivers`	Phishing scams distribute funds across many addresses
`active_days`	Fraudulent activity is often bursty and short-lived

This design improves generalization and avoids overfitting to identifiers.

Model Selection (XGBoost)

After evaluating multiple algorithms conceptually (Logistic Regression, Random Forest), **XGBoost** was selected due to:

- Strong performance on tabular datasets
- Ability to capture non-linear relationships
- Built-in regularization mechanisms
- Proven effectiveness in fraud detection tasks

The model outputs both:

- A **binary fraud prediction**
- A **fraud probability score**, providing interpretability

Challenges Faced in ML Implementation

Problem 1: Feature mismatch during inference

- **Issue**: The trained model expected features in a fixed order and format.
- **Solution**: I explicitly enforced feature column ordering during inference, ensuring consistency between training and prediction pipelines.

Problem 2: Handling skewed transaction values

- **Issue**: Ethereum transaction values are highly skewed.
- **Solution**: Applied scaling and behavioural aggregation instead of relying on raw values.

These steps stabilized the model and improved reliability.

Blockchain Design (Solidity)

Blockchain was **not used to perform fraud detection**, but rather to:

- Ensure **immutability** of fraud detection outcomes
- Provide a **verifiable audit trail**
- Prevent post-prediction manipulation

Smart Contract Architecture

The Solidity contract was designed with the following components:

- **Struct** to store:
  - SHA-256 hash of ML output
  - Fraud classification
  - Timestamp
- **Mapping** from hash → stored record
- **State-changing function** to add new records
- **View function** to retrieve stored records
- **Event emission** for transparency

Only **cryptographic hashes** are stored on-chain, ensuring data privacy and minimal gas usage.

Cryptographic Integrity via SHA-256

Before interacting with the blockchain, the ML output is hashed using **SHA-256**. The hash is generated from:

- Wallet identifier
- Fraud prediction
- Probability score
- Timestamp

This ensures:

- Any modification results in a different hash
- Blockchain stores proof, not sensitive data
- Integrity and non-repudiation

Integration of ML, API, and Blockchain

Role of the API Layer

A **FastAPI backend** was implemented to act as the **orchestration layer** between:

- User input
- ML inference
- Hashing
- Blockchain storage

FastAPI was chosen because:

- It is high-performance and lightweight
- Provides automatic request validation
- Offers Swagger UI for easy testing
- Integrates naturally with Python-based ML code

API Endpoint Design

The primary endpoint implemented was:

```
POST /predict-and-store
```

This endpoint:

1. Receives transaction behaviour features

2. Runs ML inference
3. Generates a SHA-256 hash
4. Triggers a blockchain transaction
5. Returns prediction and blockchain transaction hash

Web3.py Integration

Web3.py was used to bridge Python and Ethereum.

Key elements involved:

- **Provider**: Ganache local Ethereum network
- **Signer**: Private-key–based transaction signing
- **ABI**: Interface generated from Solidity contract
- **Contract Address**: Address of deployed smart contract

Transactions are:

1. Built
2. Signed locally
3. Broadcast to the network
4. Mined and confirmed

Integration Challenges and Debugging

Problem 1: API routing and method errors

- **Issue**: Initial `404` and `405` errors when testing endpoints.
- **Solution**: Proper use of Swagger UI (`/docs`) and correct HTTP methods (POST vs GET).

Problem 2: Web3 version incompatibility

- **Issue**: `rawTransaction` attribute error.
- **Cause**: Web3.py API change.
- **Solution**: Updated to `raw_transaction` after inspecting traceback.

Problem 3: Blockchain connection failures

- **Issue**: Transactions failed when Ganache was not running.
- **Solution**: Clear separation of ML and blockchain logic, allowing ML to function independently.

Problem 4: Smart contract reverts

- **Issue**: Duplicate hashes caused contract reverts.
- **Solution**: Included timestamps in hash generation to ensure uniqueness.

Each issue was resolved through systematic isolation of components and step-by-step debugging.

User Input

↓

Machine Learning (Fraud Detection)

↓

SHA-256 Hashing (Integrity)

↓

Blockchain (Immutable Storage)

↓

Transaction Hash Returned